30-Day Go Familiarity Roadmap

ARVIS — Build it without the language being the obstacle

Goal: Know Go well enough that when you sit down to build ARVIS, you are thinking about the engineering problem — not the syntax.

Week	Focus	ARVIS Layer
1	Go mental model, types, errors, structure	How you write Go
2	Goroutines, channels, mutexes, worker pools	Concurrency — logging, killswitch
3	I/O, streaming, Redis, PostgreSQL	Data layer — the proxy's core
4	Structure, testing, Docker, integration	The skeleton of ARVIS itself

How to use this roadmap

— Move on when you can explain the concept clearly — not when the calendar says so.

— The mini-tasks are non-negotiable. Reading without building is just entertainment.

— Tools like Zap, gRPC, and deep Redis patterns are intentionally left for project week — learn the tool when the problem demands it.

— Week 4 is unboxed — no checkboxes. It is a build, not a checklist.

WEEK 1

Consolidation & The Go Mental Model

You already know the networking layer. This week is about how Go thinks — so the rest of the roadmap lands properly.

☐ How Go is Different

- ☐ Go's philosophy: simplicity, explicitness, no magic
- ☐ The workspace: GOPATH, modules, go.mod, go.sum
- ☐ The compiler: go run, go build, go fmt
- ☐ Why Go has no exceptions — error handling as a first-class citizen

Mini-task

Write a function that returns (value, error) and handle it properly in a clean module.

Types, Structs, and Interfaces

- [ ] Primitive types, zero values, short declaration :=
- [ ] Structs as your primary data container
- [ ] Methods on structs — value vs pointer receivers (this matters)
- [ ] Interfaces: implicit satisfaction and why this is powerful

Mini-task

Define a Request struct, attach methods to it, write an interface it satisfies.

Functions, Closures, and Defer

- [ ] First-class functions, passing functions as arguments
- [ ] Closures and what they capture
- [ ] defer — what it actually does and when to use it (file cleanup, unlocking mutexes)
- [ ] Panic and recover — when and why (rarely)

Mini-task

Write a middleware using a closure that wraps a function and logs before and after.

Slices, Maps, and Memory Basics

- [ ] Arrays vs slices — how slices work under the hood (pointer, length, capacity)
- [ ] Maps — iteration order, nil maps, deletion
- [ ] When Go copies and when it doesn't — understanding pointers without fear

Mini-task

Build an in-memory rule store using a map, add/remove entries via functions.

Error Handling Patterns

- [ ] The error interface, errors.New, fmt.Errorf with %w
- [ ] Wrapping and unwrapping errors
- [ ] Sentinel errors vs custom error types
- [ ] When to return errors vs when to panic

Mini-task

Build a request validator that returns rich, wrapped errors with context.

Packages, Project Structure, and Consolidation

- [ ] How Go packages work — visibility, naming conventions
- [ ] Structuring a real project: cmd/, internal/, pkg/
- [ ] Pull together Days 1–5: refactor mini-tasks into a clean package structure

Mini-task

Reorganize your week's code into a proper multi-package project.

Rest and Review

- [ ] Revisit anything from the week that felt unclear
- [ ] Read through your own code — can you explain every line?
- [ ] Optional: Read the first chapter of The Go Programming Language (Donovan & Kernighan)

WEEK 2

Concurrency: The Heart of Go

Nothing from Python or JavaScript maps cleanly onto this. This is also the heart of what makes ARVIS fast.

Goroutines and the Go Scheduler

- What a goroutine actually is — not a thread
- go func() — spawning goroutines, the risk of fire-and-forget
- Why goroutines are cheap: stack size and the scheduler
- The problem: how do you know when a goroutine is done?

Mini-task

Spawn 10 goroutines simulating request logs. Observe the chaos without synchronization.

sync.WaitGroup and Lifecycle Control

- sync.WaitGroup — Add, Done, Wait
- The pattern: always call wg.Add before spawning, defer wg.Done inside
- Common mistakes: adding inside the goroutine, forgetting Done

Mini-task

Fix yesterday's chaos — use WaitGroup to wait for all 10 goroutines before exiting.

Channels: Communication Between Goroutines

- What channels are and why they exist: don't communicate by sharing memory
- Unbuffered vs buffered channels — the difference in behavior
- Sending, receiving, closing — what happens when you read from a closed channel

Mini-task

Build a pipeline — one goroutine generates request IDs, sends them, another reads and logs.

Select and Timeouts

- select statement — waiting on multiple channels simultaneously
- time.After for timeouts — combining with select
- default case — non-blocking channel operations

Mini-task

Build a mock killswitch — a goroutine streams tokens on one channel, a timeout fires on another. Whichever comes first wins.

Mutexes and Race Conditions

- What a race condition actually is — and why Go's race detector exists
- sync.Mutex — Lock, Unlock, always deferring Unlock
- sync.RWMutex — when reads are frequent and writes are rare (your rule store)

Mini-task

Add a mutex to your in-memory rule store. Run with -race and confirm it's clean.

Worker Pools

- The pattern: fixed goroutines reading from a shared jobs channel
- Why you don't want unlimited goroutines under load
- Graceful shutdown — closing the jobs channel to signal workers to stop

Mini-task

Build a worker pool of 5 goroutines processing 50 simulated audit log writes.

Concurrency Review and Integration

- Pull together the week: goroutines, WaitGroup, channels, select, mutex, worker pool

Mini-task

Build a concurrent audit logger — requests come in, a worker pool writes logs async, a mutex protects a shared counter, a timeout kills slow writers. This is a mini version of ARVIS's logging layer.

WEEK 3

I/O, Streaming, and the Data Layer

The networking layer you already know. This week goes deeper — into how data moves through your proxy and how ARVIS talks to Redis and Postgres.

---

io.Reader and io.Writer: The Core Abstraction

- Why everything in Go is a Reader or Writer
- io.ReadAll, io.Copy, io.TeeReader — when to use each
- bufio.Scanner and bufio.Reader for reading line-by-line or token-by-token

Mini-task

Read a large JSON file token by token using bufio, printing each token without loading the whole file into memory.

Streaming HTTP Responses

- How streaming works at the HTTP level: chunked transfer encoding
- Reading a streaming response body chunk by chunk
- http.Flusher — how to stream a response back to the client in real time

Mini-task

Build a handler that calls a mock streaming endpoint and forwards each chunk to the client without buffering the whole response.

JSON: Beyond the Basics

- json.Marshal/Unmarshal vs json.NewEncoder/NewDecoder — when to use which
- Struct tags: json:"field_name,omitempty"
- Handling unknown fields, partial decoding with json.RawMessage

Mini-task

Parse an OpenAI-style streaming response (SSE format) — extract the content field from each chunk.

Context: Cancellation and Deadlines

- context.Background(), WithCancel, WithTimeout, WithDeadline
- Passing context through your call stack — every I/O function should accept a context
- Checking ctx.Done() inside a goroutine

Mini-task

Add context cancellation to your streaming handler — if the client disconnects mid-stream, the upstream request cancels immediately.

Redis with go-redis

- Connecting to Redis, basic get/set/delete
- Redis Pub/Sub in Go — subscribing in a goroutine, receiving messages
- Using Redis for rate limiting: increment a counter with expiry

Mini-task

Build a rule update listener — subscribe to rules:update, update your in-memory rule map when a message arrives, protect the map with RWMutex.

PostgreSQL with pgx

- Connecting with pgx, connection pooling with pgxpool
- Executing queries, prepared statements, scanning rows into structs
- Inserting records asynchronously — fire and forget with a goroutine

Mini-task

Write an audit log inserter — accepts a log entry struct, inserts into Postgres asynchronously, handles errors without blocking the main request.

Rest and Review

- Review the full week
- Can you trace a request through your proxy — from incoming JSON, through streaming, through rule checking, to async log write?
- Optional: Read the net/http source for ReverseProxy — you'll understand it now

WEEK 4

Structure, Tooling, and Baby ARVIS

Pull everything together into a real Go project — and build a small but complete slice of what ARVIS needs.

Project Structure for Real Services

- The standard Go project layout: cmd/, internal/, pkg/, config/
- internal/ — why it exists and how it enforces boundaries
- Dependency injection without a framework — passing dependencies explicitly

Mini-task

Scaffold a clean ARVIS project with packages for proxy, rules, logger, config.

Configuration and Environment

- Reading environment variables with os.Getenv
- Structured config with viper or a simple config struct from a .env file
- Config validation at startup — fail fast if required values are missing

Mini-task

Add a config layer to your ARVIS scaffold — load Redis URL, Postgres DSN, and port from environment.

Structured Logging with slog (Deep Dive)

- Log levels, structured fields, JSON output
- Attaching request ID and user context to every log line automatically
- Log sampling for high-throughput proxies — don't log every token

Mini-task

Build a logger middleware that attaches a request ID to the context and includes it in every log line for that request's lifetime.

Testing in Go

- testing package basics — TestXxx functions, t.Error, t.Fatal
- Table-driven tests — the Go standard pattern
- Mocking with interfaces — why you defined that interface in Week 1
- httptest.NewRecorder and httptest.NewServer for testing HTTP handlers

Mini-task

Write tests for your rule store (add, remove, check) and for one HTTP handler using httptest.

- [ ] **The Race Detector and pprof Basics**
  - [ ] Running your full proxy under -race — fix anything that fires
  - [ ] net/http/pprof — exposing a profiling endpoint
  - [ ] Reading a CPU profile — finding where time is actually spent

Mini-task

Add a pprof endpoint to your ARVIS scaffold, run a basic load test with hey or wrk, read the profile.

- [ ] **Docker and the Go Binary**
  - [ ] Multi-stage Docker builds — builder compiles, final stage is scratch or alpine
  - [ ] Why Go binaries are special: statically compiled, tiny
  - [ ] docker-compose for local development with Redis and Postgres

Mini-task

Containerize your ARVIS scaffold — build it, run it with docker-compose alongside Redis and Postgres.

Baby ARVIS — Three Build Days

No checkboxes. This is a build.

By the end of these three days you will have a small but complete Go service that looks like a real ARVIS component. Every pattern is production-aware. You understand every line of it.

Component	What it does
POST /v1/chat	Accepts a JSON chat request and forwards it to a mock upstream as a streaming response
Token Scanning	Reads each streamed token and checks it against an in-memory rule list
Killswitch	On rule violation — cancels the stream, returns 403 with a reason, logs the event
Live Rule Updates	Updates its rule list in real time via Redis Pub/Sub — no server restart
Async Audit Logging	Writes every request to PostgreSQL asynchronously via a worker pool
/health endpoint	Checks Redis and PostgreSQL connectivity and reports status
Structured Logging	Every log line carries a request ID — using the slog middleware from Week 4
Docker	Runs cleanly with docker-compose alongside Redis and Postgres

---

This is not a toy. This is the skeleton of ARVIS.

Intelligence that never sleeps — starts here.